

Tema: Introducción a la Ingeniería de Software

Temas Abordados (Índice)

Tema 1: Evolución del software

Breve introducción histórica.

Impacto socioeconómico y el fenómeno de la "Tech Ascendancy" en la actualidad.

Tema 2: Concepto de Software e Ingeniería de Software

Definición integradora de Software.

Tipos de software.

Diferencias fundamentales con la manufactura tradicional.

Definiciones formales de la Ingeniería de Software.

Mitos del Software (Gestión, Cliente y Desarrollador).

Ética y Responsabilidad Profesional (Código ACM/IEEE).

Tema 3: Cuerpos de Conocimiento de la Disciplina

Fundamento y propósito de un Cuerpo de Conocimiento.

SWEBOK (Software Engineering Body of Knowledge).

SEBoK (Systems Engineering Body of Knowledge).

Tema 4: La Materia en Contexto

Mapa de la materia: Disciplinas Técnicas, de Gestión y de Soporte.

Profundización en áreas clave de la cátedra.

Tema 5: El Proceso de Software

Definición de proceso de software y sus tres componentes básicos.

Las 4P de la Gestión de Proyectos (Personal, Producto, Proceso, Proyecto) y su relación con los componentes del proceso.

Enfoque Definido (industrial) vs. Enfoque Empírico (adaptativo).

El Patrón de Conocimiento en procesos empíricos (La analogía de Irvine).

Ciclo de vida del proyecto vs. Ciclo de vida del producto (Modelo de evolución de Sommerville y Rajlich-Bennett).

Tema 1: Evolución del software

El origen de la Ingeniería de Software se remonta formalmente a 1968, durante una conferencia de la NATO (OTAN) en Garmisch (Alemania), donde se acuñó el término para tratar la denominada "crisis del software". En ese entonces, el desarrollo de sistemas complejos sufría de graves deficiencias: **los productos no eran confiables, costaban mucho más de lo previsto, se entregaban con retrasos extremos y no cumplían con los requerimientos reales del cliente.**

A lo largo de las décadas posteriores, la disciplina maduró a través de hitos teóricos y técnicos claves:

1975 (The Mythical Man-Month): Frederick Brooks publica su libro clásico analizando las lecciones del desarrollo del sistema operativo OS/360 de IBM y planteando límites fundamentales a la intercambiabilidad de hombres y meses en la estimación de esfuerzos (Ley de Brooks).

1978 (Análisis Estructurado): Tom DeMarco introduce las técnicas del análisis estructurado, formalizando el modelado y la especificación del flujo de información en los sistemas.

Década de 1980 (Grandes Errores): Se hacen públicos los primeros grandes errores catastróficos de software en sistemas comerciales y de defensa, evidenciando la necesidad de rigor metodológico.

1987 (No Silver Bullet): Brooks publica su ensayo clásico estableciendo que la complejidad, conformidad, variabilidad e invisibilidad son características esenciales del software y que no existe una única solución mágica para duplicar la productividad en el corto plazo.

1989 (Madurez de Procesos): Watts Humphrey publica Managing the Software Process, introduciendo la noción de madurez de procesos y sentando las bases organizacionales para el aseguramiento de la calidad.

Década de 1990 (Internet y POO): La explosión de Internet, la Programación Orientada a Objetos (POO) y la World Wide Web cambian radicalmente la arquitectura del software de sistemas monolíticos a sistemas ampliamente distribuidos.

2001 - 2003 (Enfoques Ágiles y Lean): Nace el Manifiesto Ágil y se expanden filosofías de eficiencia operativa como Lean Software Development, acelerando las entregas incrementales bajo entornos de alta incertidumbre.

En la actualidad, este desarrollo histórico se ha decantado en el fenómeno de la **"Tech Ascendancy"** (Ascendencia Tecnológica), donde la economía mundial está dominada por corporaciones cuyo valor de mercado y ventajas competitivas residen esencialmente en el software. Las diez corporaciones globales más grandes del mundo en la actualidad (como NVIDIA, Alphabet, Apple, Microsoft, Amazon, TSMC, Broadcom, Meta Platforms y Tesla) operan bajo la premisa de que el software ya no es simplemente un producto empaquetado, sino un motor de conocimiento y un servicio distribuido bajo demanda a escala planetaria (Software as a Service - SaaS).

Tema 2: Concepto de Software e Ingeniería de Software

Definición Integradora del Software

Básicamente, se entiende al software como el conjunto de programas de cómputo (instrucciones que al ejecutarse de manera correcta proveen la funcionalidad deseada), procedimientos, reglas, documentación asociada y datos de configuración.

Sin embargo, para un análisis de sistemas se requiere una definición integradora:

"El software es conocimiento representado en distintos niveles de abstracción."

El software no es meramente líneas de código ejecutables; es la destilación de **reglas de negocio, lógica y procesos conceptualizados** por los seres humanos que se materializa mediante un lenguaje de programación para interactuar con el silicio del hardware.

Tipos Básicos de Software

Software de Sistemas (System software): Programas escritos para dar servicio de infraestructura a otros programas. Tienen una interacción íntima con el hardware, operan de forma concurrente, manejan recursos compartidos y procesan datos deterministas e indeterministas (ej. sistemas operativos, compiladores, manejadores de dispositivos).

Utilitarios: Herramientas de software diseñadas para tareas específicas de optimización, diagnóstico y soporte operativo de la computadora.

Software de Aplicación: Programas aislados que resuelven una necesidad de negocio específica o procesan transacciones comerciales o técnicas en tiempo real para facilitar la toma de decisiones (ej. sistemas de información gerencial, ERP, software médico).

5 Razones para no comparar Software con Manufactura Tradicional

Intentar aplicar las métricas y controles de las líneas de producción fabril al software es un error conceptual común en la administración debido a cinco diferencias esenciales:

El software es menos predecible: Es una labor puramente intelectual e interpretativa del ser humano, donde un cambio en el equipo de desarrollo o en los requisitos del cliente puede variar drásticamente el resultado final de un proceso idéntico.

No hay producción en masa: En manufactura se calibran máquinas para producir millones de copias idénticas. En software, el esfuerzo se concentra enteramente en la creación del

"diseño único" (el desarrollo de la primera copia). Las copias posteriores se realizan mediante duplicación digital instantánea a costo marginal cero.

No todas las fallas son errores: Muchas fallas operativas provienen de discrepancias semánticas, requerimientos ambiguos o cambios dinámicos del entorno de uso, no de defectos mecánicos de construcción.

El software no se gasta (se deteriora): Mientras que los componentes de hardware sufren de desgaste físico y fatiga de materiales y se solucionan con repuestos estandarizados, el software solo se deteriora debido a los cambios continuos aplicados en su mantenimiento, los cuales degradan paulatinamente su estructura interna original.

El software no está gobernado por las leyes de la física: Al ser una entidad lógica y abstracta, el software no tiene límites naturales de espacio o materiales, lo que le permite alcanzar de forma sumamente rápida una complejidad extrema que excede la comprensibilidad humana.

Definiciones Formales de la Ingeniería de Software

David Parnas: "The multi-person construction of multi-version software" (La construcción multi-persona de software multi-versión).

Esta definición destaca los desafíos clave de la disciplina: la necesidad de coordinación del equipo y el manejo de múltiples variantes a lo largo del tiempo.

IEEE (formalizada en SWEBOK): La aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento del software; es decir, la aplicación de la ingeniería al software.

Ian Sommerville: Una disciplina de la ingeniería que se ocupa de todos los aspectos de la producción de software, desde las primeras etapas de la especificación del sistema hasta el mantenimiento del mismo después de que se pone en operación.

Conclusión Fundamental de la Unidad: Saber programar NO es suficiente. El desarrollo profesional de sistemas complejos requiere enfoques rigurosos de especificación, diseño, verificación y gestión que exceden largamente la codificación individual.

Mitos del Software (Enfoque de Pressman)

Los mitos del software son creencias erróneas pero intuitivas arraigadas en la industria que generan falsas expectativas e insatisfacción:

Mitos de Gestión:

Mito: "Tenemos un libro lleno de estándares y procedimientos; mi personal ya sabe todo lo necesario."

Realidad: La mayoría de las veces el libro está obsoleto, es inflexible o los desarrolladores ni siquiera conocen su existencia.

Mito: "Si nos atrasamos, agregamos más programadores y nos ponemos al corriente."

Realidad: Ley de Brooks: "agregar personal a un proyecto de software atrasado lo atrasó más", debido a que los desarrolladores originales deben gastar tiempo en capacitar a los nuevos, aumentando exponencialmente los canales de comunicación y coordinación.

Mito: "Si subcontratamos el desarrollo a un tercero, puedo descansar y desentenderme."

Realidad: Si una organización no sabe cómo administrar y controlar internamente sus requerimientos de software, fracasará rotundamente al gestionar contratos externos.

Mitos del Cliente:

Mito: "Una declaración general de objetivos es suficiente para comenzar a codificar; los detalles se pueden definir más tarde."

Realidad: El diseño prematuro o la codificación basada en requerimientos ambiguos e incompletos es el catalizador número uno de fallas catastróficas y reescritura de código.

Mitos del Desarrollador (Profesional):

Mito: "Una vez que escribimos el programa y hacemos que funcione, nuestro trabajo ha terminado."

Realidad: Entre el 60% y 80% del esfuerzo total de la vida de un software ocurre después de que se entrega al cliente por primera vez (fase de mantenimiento y evolución).

Mito: "Hasta que el programa no esté corriendo, no hay forma de evaluar su calidad."

Realidad: Las revisiones técnicas formales de los modelos de requerimientos y diseño son "filtros de calidad" mucho más eficientes y económicos para detectar defectos tempranos que las pruebas dinámicas.

Mito: "El único producto de trabajo que entregamos es el programa ejecutable."

Realidad: Una configuración de software exitosa incluye modelos, especificaciones, planes, manuales de usuario y datos de prueba que guían el soporte y los cambios futuros.

Ética y Responsabilidad Profesional

Como profesionales de la ingeniería de sistemas, el trabajo realizado impacta de manera profunda en la privacidad, la seguridad y el bienestar público. Por ello, se debe trabajar bajo un estricto compromiso ético formalizado por la ACM y la IEEE Computer Society en su Código de Ética y Práctica Profesional. Este código define lineamientos organizados bajo ocho principios fundamentales:

Público: *Actuar de manera coherente con el interés público*

Cliente y Empleador: *Actuar en función del mejor interés de sus clientes y empleadores, compatible con el interés público.*

Producto: *Garantizar que sus productos y modificaciones relacionadas satisfagan los más altos estándares profesionales posibles.*

Juicio: *Mantener integridad e independencia en su juicio profesional.*

Administración: *Los líderes deben suscribirse y promover un enfoque ético en la gestión del desarrollo y mantenimiento del software.*

Profesión: *Promover la integridad y reputación de la profesión, compatible con el interés público.*

Colegas: *Ser justos y brindar apoyo mutuo en el equipo.*

Uno mismo: *Participar en el aprendizaje continuo sobre la práctica de su profesión y promover un enfoque ético.*

Tema 3: Cuerpos de Conocimiento

Un Cuerpo de Conocimiento (Body of Knowledge) no debe confundirse con la literatura bibliográfica en sí. La bibliografía y las prácticas acumuladas durante décadas constituyen el conocimiento de la profesión; el cuerpo de conocimiento actúa como una guía consensuada y validada que demarca los límites de la disciplina, clarifica la frontera con ciencias relacionadas y estructura qué áreas de estudio debe dominar un profesional universitario. Sin un cuerpo de conocimiento formal, resulta imposible validar planes de estudio de grado o estructurar exámenes de acreditación profesional.

SWEBOK (Software Engineering Body of Knowledge)

Mantenido e impulsado por la IEEE Computer Society. El SWEBOK busca proveer una caracterización consensuada de la ingeniería de software a nivel mundial, marcando su diferencia con las ciencias de la computación y de sistemas.

La versión 3.0 de la IEEE está conformada por **15 áreas de conocimiento (KAs)**. El SWEBOK se enfoca en delimitar el conocimiento "generalmente aceptado", caracterizado como las prácticas tradicionales y maduras que son útiles y aplicadas en la mayoría de los proyectos la mayor parte del tiempo, con un amplio consenso de la industria y la academia (excluyendo prácticas experimentales o de investigación avanzada).

SEBoK (Systems Engineering Body of Knowledge)

Es la guía del cuerpo de conocimiento de la Ingeniería de Sistemas, la cual adopta una perspectiva más holística y sociotécnica. No se limita únicamente al software, sino que se ocupa de coordinar de manera integrada el hardware, el software, los procesos operativos y el personal humano de la organización para lograr un objetivo estratégico.

La versión de la SEBoK (BKCASE) se estructura en partes fundamentales, que incluyen:

Fundamentos de la Ing. de Sistemas

Pensamiento sistémico, representación mediante modelos, ciencia de sistemas.

Ingeniería de Sistemas y Gestión

Modelos de ciclo de vida, planificación, control, estándares.

Aplicaciones de la Ingeniería de Sistemas

Ingeniería de sistemas de productos, servicios, empresas y sistemas de sistemas.

Facilitadores de la Ingeniería de Sistemas

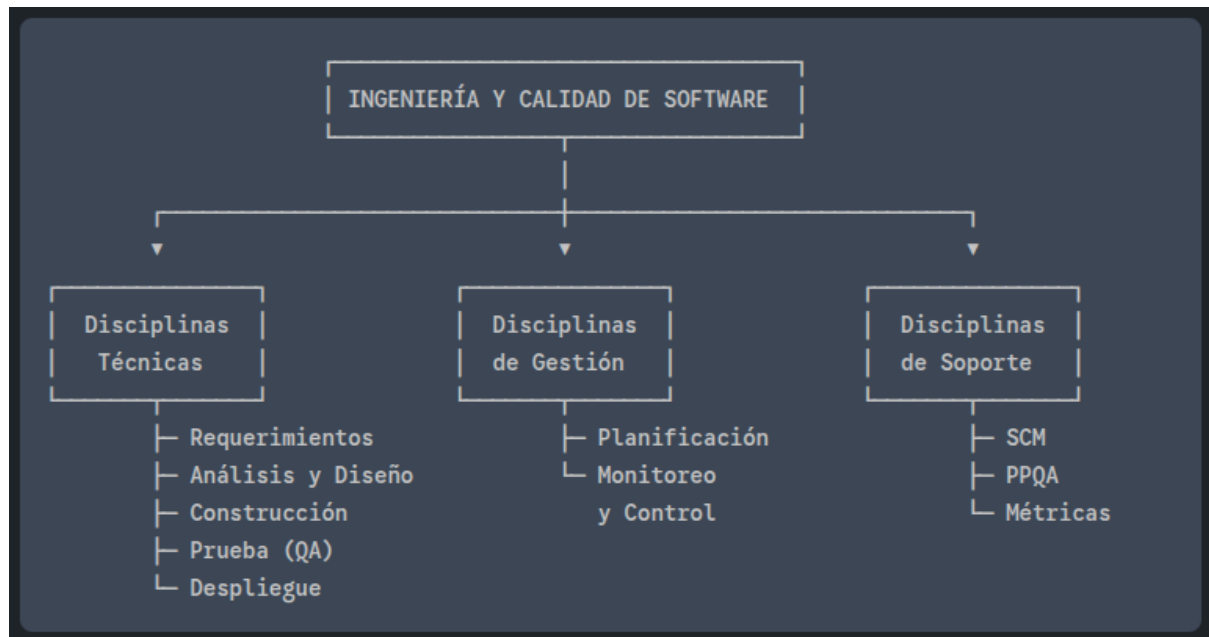
Habilitar equipos, individuos y empresas.

Disciplinas Relacionadas

Intersección crítica entre Ing. de Sistemas, Ing. de Software e Ing. Industrial.

Tema 4: La Materia en Contexto

La cátedra organiza el mapa del desarrollo profesional de software en tres grupos de disciplinas que interactúan entre sí de manera continua:



Profundización en las Áreas Clave de la Materia

Prueba (Testing / QA): Proceso crítico e iterativo para evaluar dinámicamente un sistema ejecutable. La cátedra profundiza en que las pruebas no deben ser vistas como una simple fase tardía ("red de seguridad"), sino que el diseño de casos de prueba debe comenzar desde la ingeniería de requerimientos para guiar el desarrollo libre de defectos.

Despliegue (Deployment): Conjunto de flujos de trabajo orientados a empaquetar, distribuir, instalar, configurar y entregar de forma segura la versión funcional de un incremento del software en la plataforma de ejecución real del cliente.

Planificación de Proyecto: Estimación sistemática y empírica de esfuerzos, plazos, costos y recursos requeridos. Se enfoca en mitigar riesgos y fijar metas de calendario realistas para evitar la fatiga y el fracaso del equipo bajo presión de entregas.

Monitoreo y Control de Proyectos: Seguimiento cuantitativo riguroso del progreso real del desarrollo contra el plan de actividades inicial, utilizando técnicas de control basadas en puntos de referencia clave para corregir desviaciones tempranas.

Gestión de Configuración de Software (SCM): Actividad sombrilla vital para administrar los efectos del cambio y garantizar la trazabilidad de los componentes. Abarca:

- **Administración del cambio:** Evaluar e implementar solicitudes de cambio de manera controlada.

- **Gestión de versiones:** Controlar las diferentes variantes e iteraciones de los Ítems de Configuración de Software (ICS).
- **Construcción del sistema:** Proceso automatizado de compilar y enlazar componentes para generar un sistema ejecutable.
- **Aseguramiento de la Calidad (PPQA / QA):** Actividad sistemática orientada a definir y verificar que el proceso de desarrollo cumpla con los estándares organizacionales e internacionales de calidad. Su pilar clave en la cátedra son las Revisiones Técnicas aplicadas a requerimientos, diseños y código para filtrar errores tempranos de forma económica.
- **Métricas:** El uso de la medición directa e indirecta del proceso, el proyecto y el producto de software como la única herramienta objetiva de ingeniería para valorar la productividad, la complejidad y la calidad, guiando la toma de decisiones y la mejora continua del proceso.

Tema 5: El Proceso de Software

Un Proceso de Software es un conjunto estructurado y coherente de actividades, métodos, prácticas y transformaciones de trabajo ejecutadas con la finalidad de desarrollar, mantener y evolucionar un sistema de software de alta calidad.

Los Tres Componentes Básicos del Proceso

De acuerdo al modelo de ingeniería de software, para que un proceso sea viable y exitoso, se requiere una sinergia equilibrada de tres elementos:

Personas con habilidades, entrenamiento y motivación: El factor humano es el pilar central de la ingeniería; sin personal calificado y autodirigido, el proceso falla.

Procedimientos y métodos: Definen el "cómo" y el "cuándo" de las actividades técnicas (ej. especificación de requerimientos, modelado del diseño, pruebas).

Herramientas y equipos: El apoyo automatizado o semiautomatizado que asiste al personal de desarrollo en el manejo de la complejidad y la información de la configuración del sistema.

El Concepto de las 4P de la Gestión de Proyectos (Pressman)

Para que un proceso de desarrollo se ejecute con éxito dentro de un contexto organizacional, la gestión del software debe enfocarse de manera equilibrada en cuatro variables críticas (las 4P):

Personal (People / Persona): El factor humano es el elemento esencial y más influyente en el éxito del software. Consiste en organizar equipos de trabajo competentes, motivados, coordinados y con un liderazgo enfocado en la resolución de problemas.

Producto (Product): Representa el resultado que se desea construir. Antes de planificar el proyecto, se deben establecer de manera clara los objetivos y el alcance de este producto, considerando alternativas técnicas y limitaciones del negocio (descomposición del problema).

Proceso (Process): Proporciona la estructura conceptual (las actividades de marco de trabajo y actividades sombilla) sobre la cual se monta el plan de desarrollo. Debe adaptarse de manera flexible según el equipo de trabajo y las características del producto.

Proyecto (Project): Es el contenedor temporal que agrupa al personal, al proceso y al producto. Implica todas las tareas de planificación, monitoreo, control de riesgos y calendarización para asegurar que el equipo trabaje a su máxima potencia y el desarrollo llegue a buen puerto.

Relación entre las 4P y los Componentes del Proceso de Software

Las 4P de la gestión se relacionan directamente y de manera bidireccional con los conceptos de procesos y ciclos de vida abordados en la cátedra:

Personal $\longleftrightarrow$ **Personas con habilidades, entrenamiento y motivación:** El factor humano es el pilar central del proceso. Así como en las 4P el "Personal" es fundamental, el proceso de software empírico asume que las dinámicas de equipo, el aprendizaje colaborativo (sociología del grupo) y la motivación determinan la calidad del producto, por encima de las herramientas rígidas.

Proceso $\longleftrightarrow$ **Procedimientos y Métodos (Enfoque Empírico vs. Definido):** El "Proceso" de las 4P se materializa a través de los procedimientos técnicos de la organización. Al contrastar el Enfoque Definido (rígido y prescriptivo) con el Enfoque Empírico (adaptativo), se evidencia que el proceso ágil moderno da autonomía al "Personal" para inspeccionar y adaptar las actividades, promoviendo equipos autoorganizados.

Producto $\longleftrightarrow$ **Ciclo de Vida del Producto:** El "Producto" es la entidad lógica que evoluciona a largo plazo [15]. Su definición de alcance se relaciona directamente con el Ciclo de Vida del Producto (Desarrollo inicial, Evolución, Servicio y Retiro gradual). Conforme el producto evoluciona en el mercado, se aplican cambios técnicos controlados que evitan su degradación prematura.

Proyecto $\longleftrightarrow$ **Ciclo de Vida del Proyecto:** El "Proyecto" es el esfuerzo acotado temporalmente para entregar incrementos funcionales de software. Se mapea directamente en el Ciclo de Vida del Proyecto, organizándose en fases de ingeniería dinámicas (ej. Concepción, Elaboración, Construcción, Transición) para mitigar riesgos de manera cuantificable y asegurar la entrega oportuna.

Procesos Definidos vs. Procesos Empíricos

La madurez de la disciplina contrastó dos visiones filosóficas del desarrollo de software:

Característica	Enfoque Definido (Industrial)	Enfoque Empírico (Adaptativo)
Inspiración	Líneas de montaje industrial y producción en masa	Gestión de la complejidad y adaptabilidad ante la incertidumbre
Premisa	Asume que el proceso de diseño se puede planificar por completo de manera determinista e inflexible de antemano.	Reconoce que el diseño de software es creativo, dinámico y que es imposible conocer todos los detalles al inicio.

Característica	Enfoque Definido (Industrial)	Enfoque Empírico (Adaptativo)
Control	Se mide el avance estricto contra un plan preestablecido (Plan-driven).	Basado en la inspección constante, el aprendizaje social y la adaptación continua (Agile).
Manejo del Cambio	El cambio se ve como una falla o interrupción que debe evitarse y minimizarse.	El cambio se ve como algo natural que agrega valor real al cliente a través de ciclos de retroalimentación cortos.

El Patrón de Conocimiento en Procesos Empíricos (La analogía de Irvine)

Para comprender los procesos empíricos, se utiliza la famosa analogía ocurrida en el campus de la Universidad de California en Irvine:

"En lugar de diseñar y construir senderos de hormigón rígidos desde el primer día, las autoridades de la universidad simplemente sembraron césped por todo el campus y esperaron un año. Pasado ese tiempo, observaron de manera natural dónde los estudiantes habían desgastado el césped al hacer sus recorridos habituales (los 'caminitos'). Finalmente, pavimentaron exactamente esos caminos desgastados."

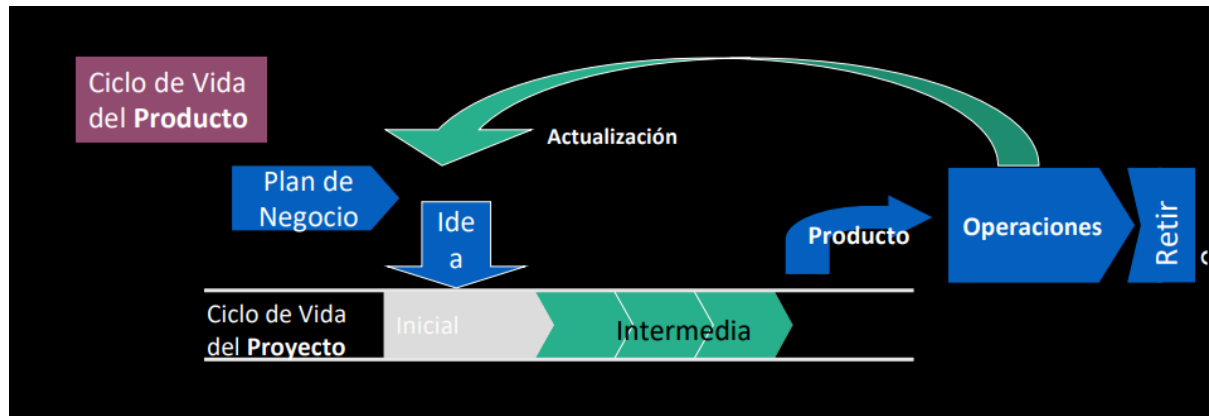
En la ingeniería de software empírica, este patrón de conocimiento establece que el proceso debe adaptarse a la realidad operativa del uso humano y la retroalimentación real del entorno, en lugar de intentar forzar de manera prescriptiva y rígida un comportamiento teórico que ignora la interacción real de los usuarios con el producto.

Ciclo de Vida del Proyecto vs. del Producto

Es fundamental para un ingeniero de sistemas diferenciar ambos términos:

Ciclo de Vida del Proyecto: Es una entidad acotada en el tiempo que abarca desde la aprobación del caso de negocio inicial hasta la entrega final del incremento funcional y despliegue del sistema. En marcos de proceso como RUP o cascada clásica, se organiza a través de fases dinámicas bien delimitadas (ej. Concepción, Elaboración, Construcción y Transición o Análisis, Diseño, Codificación, Prueba y Mantenimiento).

Ciclo de Vida del Producto: Es un proceso continuo y de muy largo plazo (a menudo superior a una década) que abarca la existencia total del software en el mercado. Sommerville y Rajlich-Bennett integran la dinámica del cambio del producto en un modelo evolutivo continuo compuesto por cuatro etapas que se traslapan:



Desarrollo Inicial: Fase creativa de concepción, análisis, diseño e implementación donde se lanza la versión 1 del sistema operativo.

Evolución: Fase donde el producto está en uso exitoso y se modifican e incorporan de manera constante requerimientos funcionales significativos, adaptándose a las variaciones dinámicas del negocio y del hardware.

Servicio: Punto de transición donde la estructura interna del software se ha degradado tanto debido a los continuos cambios acumulados (Leyes de Lehman) que realizar modificaciones grandes ya no es rentable para la organización. Durante esta etapa, el software sigue operativo, pero solo se realizan mantenimientos correctivos mínimos de supervivencia y parches mientras se planifica su sustitución final.

Retiro Gradual (Phase-out): El software se retira del mercado. No se le realiza ningún tipo de modificación, por lo que los usuarios deben convivir con cualquier fallo remanente mientras migran paulatinamente hacia los nuevos sistemas de reemplazo.